
unilasalle RS
50 anos de vidas

Algoritmos e Programação

Aula 12 –Ponteiros em Python



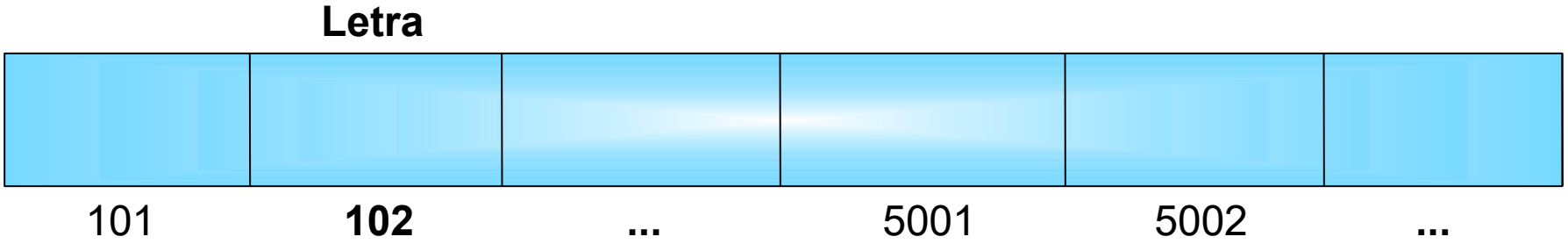
Dados do Professor

Prof M.e Rafael Gastão – Graduado em Ciência da Computação e Mestre em Dados e IA pela UFRGS.
rafael.ferreira@unilasalle.edu.br

Principio de Ponteiros

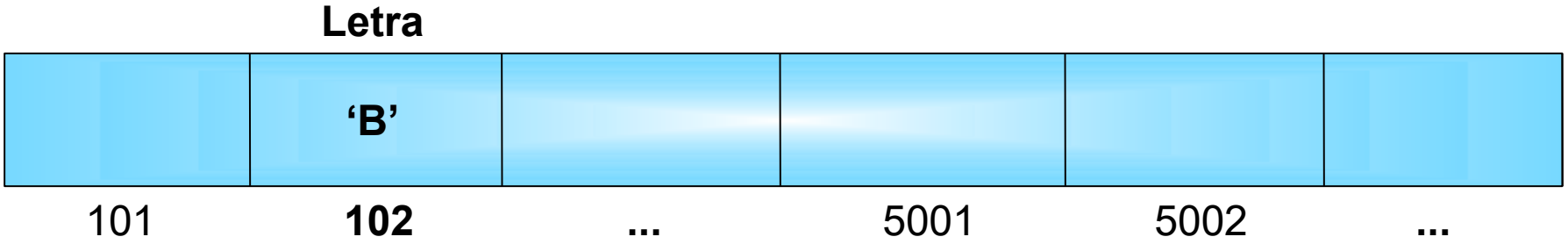
Principio de Ponteiros

- Em programação, uma variável é nada mais que uma posição livre da memória;
- Ao criarmos a variável char **Letra** esta passa a ocupar um byte da memória. O endereço, neste exemplo, é **102**;
- Para o programador é mais simples utilizar o nome da variável, no caso **Letra.**



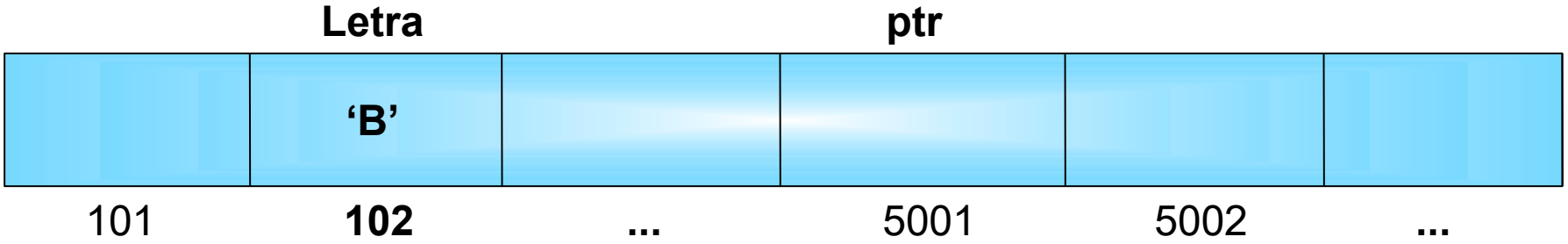
Principio de Ponteiros

- O comando Letra = 'B'; diz ao computador que o caractere 'B' deve ser armazenado no endereço 102



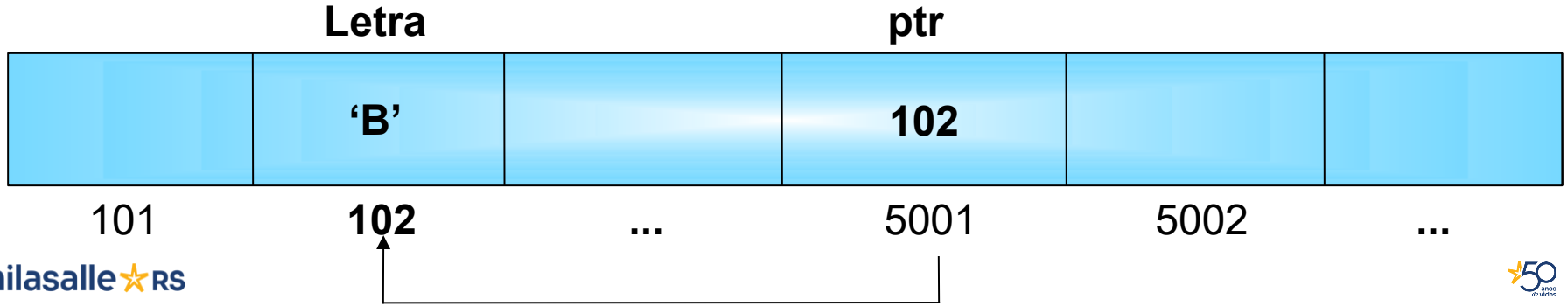
Principio de Ponteiros

- **Ponteiro**: é um tipo de variável que “aponta” para um outro endereço de memória;
- Supondo a existência de um ponteiro chamado **ptr;**
- Ele também é uma variável e ocupa uma posição de memória.



Principio de Ponteiros

- A variável **ptr** é um ponteiro;
- Para **ptr** apontar para a variável **Letra**, devemos usar o operador **&;**
- **ptr** = **&**Letra;



Principio de Ponteiros

Sintaxe:

tipo * **ptr** , sendo que:

- **ptr**: nome da variável;
- **tipo** : tipo da variável para qual apontará;
- * : indica que é uma variável tipo ponteiro.

A carga inicial se faz com o operador &

```
int x =5;
int * ptr_x = &x;
```

Principio de Ponteiros

Exemplo:

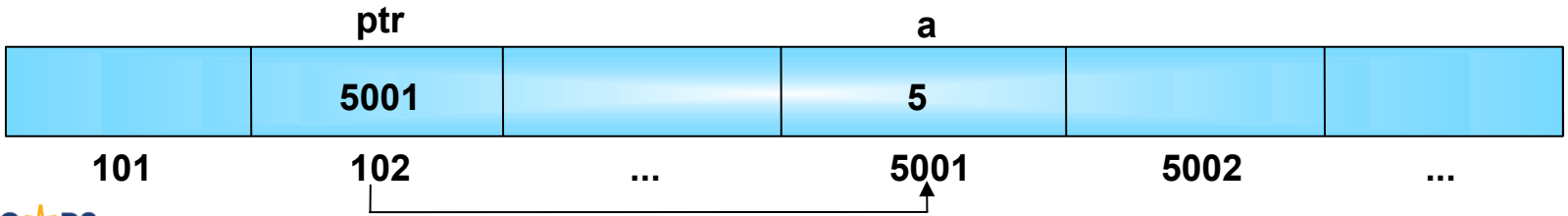
```
int a=5, b=7;
int *ptr = NULL;      (ptr não aponta para nada)
ptr = &a;
```

Valor das variáveis:

a tem valor: 5

ptr tem endereço da variável a: 5001

*ptr representa o valor da variável a que é: 5



Ponteiros em Python

Ponteiros em Python

No Python padrão, não declaramos ponteiros explicitamente, porque toda variável já é um ponteiro (uma referência).

```
# Exemplo 1: Variáveis como Referências (Ponteiros implícitos)

# Criamos uma variável 'a' apontando para o número 10
a = 10

# Dizemos que 'b' recebe 'a'
b = a

# Vamos imprimir os valores e os "endereços de memória" (IDs)
print(f"Valor de a: {a} | Endereço (ID) de a: {id(a)}")
print(f"Valor de b: {b} | Endereço (ID) de b: {id(b)}")

# Note que o endereço é EXATAMENTE O MESMO!
# O Python não copiou o número 10. Ele apenas fez o 'b' apontar para o mesmo lugar que o ‘a’.
```

Tipos Imutáveis em Python

Tipos Imutáveis em Python

- Diferente do C, onde você poderia usar um ponteiro para alterar o valor original, no Python, tipos básicos como Inteiros, Floats e Strings são imutáveis;
- Isso significa que, se você tentar alterar o valor de **b**, o Python não vai alterar o **10** que está na memória;
- Em vez disso, ele vai criar um novo objeto com o novo resultado e fazer o b apontar para ele;
- O **a** continuará apontando para o **10** original.

Tipos Imutáveis em Python

```
# Exemplo 2: O que acontece quando tentamos alterar o valor?

# Vamos somar 5 na variável 'b'
b = b + 5

print("\n--- DEPOIS DA ALTERAÇÃO ---")
print(f"Valor de a: {a} | Endereço (ID) de a: {id(a)}")
print(f"Valor de b: {b} | Endereço (ID) de b: {id(b)}")

# Observe que o 'a' continua intacto e mantendo o mesmo endereço original.
# O 'b' agora tem um novo valor e APONTA PARA UM NOVO ENDEREÇO de memória.
```

Porque usar Ponteiros?

Porque usar Ponteiros?

Eficiência de Memória: Quando você faz x = y, o Python não copia o objeto inteiro (o que gastaria tempo e processamento). Ele apenas cria uma nova referência apontando para o mesmo lugar. Isso é essencial quando estamos lidando com grandes volumes de dados;

Prevenção de Vazamento de Memória (Memory Leaks): No lugar de alocar (malloc) e liberar (free) memória manualmente, o Python usa um Garbage Collector (Coletor de Lixo). Se um objeto na memória não tem mais nenhuma "etiqueta" apontando para ele, o Python o destrói automaticamente;

Porque usar Ponteiros?

Segurança: Evita o famoso erro de Segmentation Fault ou acesso indevido à memória (como tentar acessar o endereço de um ponteiro nulo ou fora dos limites do array), tornando os sistemas mais robustos e o código mais legível;

Abstração: Permite que o programador foque na lógica do algoritmo em si (como manipular matrizes, treinar modelos, criar regras de negócio) e não na microgestão do hardware.

Exemplos Práticos

Exemplos Práticos

```
# Criamos um texto longo na memória
texto_original = "Um texto muito grande que ocuparia muito espaço se fosse copiado várias vezes..."

# Ao fazer isso, não estamos copiando o texto.
# Estamos apenas criando um novo ponteiro apontando para ele.
texto_backup = texto_original

if id(texto_original) == id(texto_backup):
    print("Sucesso! As duas variáveis apontam para a exata mesma memória.")
    print("O Python economizou RAM pois o texto não foi duplicado!")
```

Exemplos Práticos

```
# Inicializando duas variáveis
x = 99
y = 55

print("--- ANTES DA TROCA ---")
print(f"X aponta para: {x} (Endereço: {id(x)})")
print(f"Y aponta para: {y} (Endereço: {id(y)})")

# Swap no Python: Trocando os ponteiros de lugar em uma única linha!
# O X vai apontar para onde o Y apontava, e o Y vai apontar para onde o X apontava.
x, y = y, x

print("\n--- DEPOIS DA TROCA ---")
print(f"X agora aponta para: {x} (Novo Endereço: {id(x)})")
print(f"Y agora aponta para: {y} (Novo Endereço: {id(y)})")

# Os números 99 e 55 NUNCA saíram do lugar na memória do computador.
# Foram apenas os nossos ponteiros 'x' e 'y' que cruzaram seus endereços!
```



Bons Estudos!!!



unilasalle RS 50 anos de vidas